ShopOS

MASTER PRODUCT REQUIREMENTS DOCUMENT

Complete Offline-First Shop Management for Android

Design reference: the approved pastel lavender, pearl-white and blush UI mockup
Build target: practical, reliable, zero-budget MVP with production-minded foundations

Version 1.0 | 23 October 2026

0. Instructions to the AI Studio Builder

This document is the master specification. Implement the app from it carefully and incrementally. Do not treat it as loose inspiration.

- Use the supplied ShopOS UI mockup as the PRIMARY visual reference. Reproduce its overall visual language, screen composition, proportions, palette, card treatment, icon scale, typography hierarchy, navigation, and spacing as closely as possible. Do not invent a generic dashboard or redesign the app into a different style.
- Do not replace the reference with a generic Material template, default scaffold, stock dashboard, oversized cards, generic gradients, or a random collection of rounded rectangles. Material 3 may be used as an implementation foundation, but the final UI must be visually tuned to the reference.
- All screens must be genuinely functional. Buttons, filters, tabs, search fields, save actions, payment choices, charts, menus, and navigation must do the action they visually promise. No dead controls, fake data presented as real, placeholder-only screens, or decorative interactions.
- Use realistic demo data only in a clearly labelled demo/sample state. Once the user starts using the app, show their actual locally stored data. Never silently seed fake business transactions into a real shop.
- Do not implement every phase in one uncontrolled code-generation pass. First inspect the existing project, identify its framework and current state, then implement in phases. Preserve working features and existing user data. Do not wipe the database, replace the project, or reset schemas as a shortcut.
- Do not add login, Google sign-in, paid APIs, required cloud services, subscriptions, or AI features to the MVP. The app must work without an account and without internet for core operations.
- Use English as the default language initially, but build complete Bangla localization infrastructure from the beginning. All user-facing strings must come from resources/localization, not hard-coded screen text.

1. Product Summary

ShopOS is an offline-first Android application that helps owners manage the day-to-day operation of many kinds of small and medium retail shops from one device. It combines product catalog, stock, sales/POS, receipts, customer and supplier ledgers, purchases, expenses, cashbook, reports, settings, backup and restore.

Decision	Requirement
Product	ShopOS (working name; do not hard-code brand so it can be changed globally)
Platform	Android phone first; tablet responsiveness is a bonus, not a separate product
Connectivity	Offline-first. Core business operations must never require a network connection
Budget	Zero-budget build. Use free/open-source libraries with compatible licences; no paid API dependency
Target shops	General retail: grocery, clothing, shoes, electronics, cosmetics, household goods and similar shops
Account	No mandatory account, Google login or sign-up
Primary language	English initially; Bangla supported throughout the product
Primary success metric	A shop owner can record a sale quickly and trust that stock, cash, dues and reports reconcile correctly

2. Product Principles

- Accuracy before decoration: money, stock, ledgers and auditability must be correct.
- Fast daily workflows: common actions should take minimal taps and support one-handed use.
- Progressive complexity: simple defaults for general shops; advanced product attributes are optional.
- Offline by design: local database is the source of truth in MVP.
- Explicit financial state: sales, cash received, profit, receivables and payables must not be conflated.

- Safe correction: completed transactions are reversed/corrected through traceable actions, not silently overwritten.
- Visual consistency: every screen follows one design system and the approved mockup.
- Accessible clarity: readable contrast, scalable text, touch targets, meaningful labels and understandable errors.

3. Design Reference and Visual Contract

The approved mockup is the visual source of truth: a premium, soft, modern interface with pearl/ivory backgrounds, pale lavender and muted periwinkle accents, subtle blush and cool grey surfaces, dark navy typography, restrained shadows, fine borders, and generous but controlled whitespace.

3.1 Exact visual direction

- Palette: warm pearl-white/off-white base; mist lavender and muted periwinkle as signature accents; blush only for small secondary highlights; pale neutral grey for surfaces; dark ink/navy for text. Avoid green as the dominant brand colour. Do not use saturated neon colours.
- Brand: simple abstract ShopOS mark with a soft lavender/periwinkle treatment. Keep the logo, icon and app name consistent across splash, onboarding, settings and receipts.
- Cards: light surfaces with delicate border/shadow, subtle radius, and clear internal hierarchy. Use card containers only when they group related information; do not put every line of text in its own card.
- Typography: modern sans-serif such as a system sans / Roboto with carefully tuned weights. Use strong, compact page titles; large figures only for key KPIs; small muted metadata; avoid overly tiny text.
- Icons: one consistent line-icon family, matching stroke weight and optical size. Avoid mixing emoji, filled clip-art and multiple icon styles.
- Charts: minimal and legible, with restrained lavender/periwinkle series, thin grid lines, clear labels and no decorative 3D effects.
- Motion: quick, polished transitions, subtle press feedback, smooth bottom sheets, list updates and chart entrance. Motion must be restrained and performant on lower-end Android devices.
- Texture: optional very subtle abstract translucent shapes on splash/onboarding only. Main business screens should remain calm, clean and information-first.
- Do not use a heavy glassmorphism effect on every screen. Premium feel should come from spacing, typography, composition, fine detail, and coherent motion—not blur overload.

3.2 Non-negotiable screen fidelity

- Use the mockup's screen-by-screen layout as the starting point for Splash, Dashboard, Products, New Sale, Receipt, Add Product, Inventory, Customers, Suppliers, Expenses, Reports, Sales Report, Profit & Loss, Settings and More.
- Keep the same overall screen density and relative hierarchy. Do not make the UI significantly more crowded, dramatically more spacious, or card-heavy than the reference.
- Use the same bottom navigation structure: Home, Products, Sales, Reports, More. Secondary sections such as Inventory, Customers, Suppliers, Expenses, Settings and backup are reachable through the relevant flows or More menu.
- Recreate the reference's segmented filters, compact list rows, small product thumbnails, rounded search fields, restrained floating action button, clean receipt and simple report visualisations.
- Do not copy text or numerical values from the mockup as live data. Its numbers and product labels are illustrative only.
- Before implementing all screens, build the design system and 3 representative screens, compare them against the mockup, and tune spacing/typography/radii before scaling to the remaining screens.

4. Critical Android Top System Bar / Header / Safe-Area Requirements

This section is critical. AI Studio-generated Android UI frequently places page titles, toolbars or the entire content area too far down the screen, leaves excessive blank space under the status bar, double-applies top padding, or puts controls behind the system status bar. These defects are not acceptable.

4.1 Required edge-to-edge and inset behaviour

- Inspect the actual Android project and its Compose/Activity setup before changing system-bar behaviour. Do not guess based on a web mockup.
- Use a single, deliberate edge-to-edge strategy compatible with the project's target SDK and Android version. Do not combine multiple inset-padding strategies that apply the same top inset twice.
- Account correctly for the status bar, display cutout/punch-hole, gesture navigation area and three-button navigation. Content must never be obscured by system icons or the camera cutout.
- Use WindowInsets / safe-drawing insets in the correct place. The root container, screen scaffold and individual toolbar must have a documented inset responsibility. Apply top inset exactly once.
- Do not hard-code a large top spacer, arbitrary top margin, fixed status-bar height, or repeated status-bar padding to make a screenshot appear safe.
- Do not place a second fake status bar inside the app. Android's actual status bar must remain visible and aligned with the reference's neutral background treatment.
- Toolbar/page header should begin immediately below the real status-bar safe area, with only the intentional compact spacing shown in the mockup. It must not float far down the screen.
- Do not place the page title under the status bar, behind the punch-hole, or clipped by the top screen edge. Leading back button, title and trailing actions must share a consistent horizontal baseline.
- Handle display cutouts and different aspect ratios without shifting the header to an arbitrary position. Test both gesture navigation and three-button navigation.
- Ensure the bottom navigation and bottom action buttons do not overlap the navigation bar, keyboard, or system gesture area. Use bottom insets once and adapt when the keyboard is visible.
- On screens with forms, the keyboard must not hide Save/Complete Sale buttons or the active input. Use appropriate IME padding, scroll-to-focused-field behaviour, and sensible resize handling.

4.2 Header layout contract

Element	Rule
Root screen	Full available viewport; background extends naturally behind system bars only where intended
Status bar	Use real system status bar; consistent icon contrast and background
Top safe inset	Applied once at the chosen root/scaffold layer; never repeated at child level
Toolbar	Compact and consistent height; aligned back/menu action, title and optional trailing action
Title baseline	Visually consistent across every secondary screen
Horizontal padding	One shared token, typically 16–20 dp depending on width; do not vary randomly per screen
Top content gap	Small intentional gap after toolbar; no oversized blank strip
Scrolling	Toolbar behaviour consistent: fixed header with scrolling content unless the reference clearly indicates otherwise
Bottom safe inset	Handled once; bottom navigation/action controls remain tappable and visible

4.3 Mandatory top-header validation

- Capture screenshots of Splash, Dashboard, Products, New Sale, Add Product, Reports and Settings on at least one compact phone and one tall phone/emulator.
- Check the first visible pixel row below the status bar. There must not be an unexplained blank region caused by nested padding.
- Check that toolbar top, page-title baseline, left/right margins and trailing icons align across secondary screens.
- Test portrait orientation, large font scale, display cutout and both navigation modes.
- If the header is too low, trace the inset/padding source and remove the duplicate source. Do not compensate by applying negative offsets or arbitrary per-screen hacks.
- Do not mark a screen complete until visual comparison confirms that header placement and content density match the reference.

5. Information Architecture and Navigation

Primary destination	Purpose	Key actions
Home	At-a-glance business summary	Start sale, add product, collect due, add expense; open alerts
Products	Product catalog and product setup	Search, filter, add/edit product, view product details
Sales	POS and sales history	New sale, open past sale, receipt, return/void with reason
Reports	Business performance	Sales, purchases, expenses, cash, gross profit and net profit
More	Secondary management and configuration	Inventory, Customers, Suppliers, Cashbook, Settings, Backup/Restore, Help

Navigation state must be preserved where appropriate. Returning from product details should return to the same search/filter context. Back navigation must not accidentally discard a draft sale or unsaved form; show a clear confirmation when leaving would lose work.

6. Detailed Screen Requirements

6.1 Splash and first launch

- Display the ShopOS logo, wordmark and short tagline on a pearl-white background with subtle lavender abstract forms. Match the approved mockup's calm premium atmosphere.
- Keep splash lightweight and brief; do not add a fake loading percentage or long animation.
- On every app launch, show only the platform-appropriate launch screen behaviour; do not force a lengthy onboarding sequence every time.
- First launch routes to a short setup flow: shop name, shop type (optional), currency (BDT default), language, starting cash (optional) and initial data choice.
- Do not request login or network permission. Ask only for permissions when the user invokes a feature that requires them.
- Allow skipping optional setup fields and completing setup later.

6.2 Dashboard / Home

- Top header: compact greeting/shop name, date/context and small notification/profile or business-switching affordance only if implemented.
- Primary KPI: Today's sales. Show value, comparison only when valid historical data exists, and a small restrained chart.

- Secondary KPI tiles: Gross/total profit, cash in hand, due to collect, due to pay. Values must come from the database and use consistent date boundaries.
- Low-stock alert: number of low-stock products and a short list. Tapping opens filtered inventory.
- Quick actions: New Sale, Add Product, Collect Due, Add Expense. All actions must navigate to working flows.
- Recent activity: recent sales, payments or stock changes with timestamp and amount; show an empty state when no data exists.
- First-run state: friendly empty dashboard with actions to add a product or record first sale. Never show fabricated sales figures.
- Date calculations must use local time and business timezone assumptions. Today's report must not mix yesterday's late-night records.

6.3 Products list

- Search by product name, SKU or barcode; search should be responsive and case-insensitive where appropriate.
- Category filter chips/segmented controls; filter state must visibly update the list and be preserved when navigating back.
- Each row: small image/placeholder, name, unit/variant where useful, selling price, stock quantity and low-stock indicator when applicable.
- Provide Add Product action, product details, edit, archive and stock adjustment. Avoid destructive deletion of products with transaction history.
- Support empty, loading, no-search-results and error states. In offline MVP, do not show fake network loading.

6.4 Add/Edit Product

- Fields: product name (required), category, unit, purchase cost, selling price, opening/current stock, low-stock threshold, SKU (optional), barcode (optional), notes (optional), image (optional).
- Optional attributes should be extensible: size/colour for clothing, serial/IMEI for electronics, batch/expiry for goods that need it. These advanced workflows may be staged after the core MVP.
- Validate non-empty name, valid non-negative prices, valid quantity and sensible units. Show inline messages next to the field; do not rely on generic toast-only errors.
- Purchase cost and selling price must be distinct. Margin can be shown as a preview but must not be treated as guaranteed realized profit.
- Save is disabled while invalid or while a save operation is in progress. On success, show clear confirmation and return to the appropriate previous context.
- Editing a product's current price must not change the historical price stored in old sale items.

6.5 New Sale / POS

- Provide fast product search and barcode entry/scan entry point. Camera permission is requested only when the user chooses camera scanning.
- Adding a product creates/updates a cart line with quantity controls. Prevent quantities that violate configured stock policy.
- Show line item name, unit price, quantity, line total and remove/edit affordance. Avoid tiny controls.
- Support item-level or cart-level discount only with explicit rules; never allow discount calculations to create invalid negative totals.
- Checkout summary must show subtotal, discount, total, paid amount, remaining due and payment method(s).
- Payment methods should be configurable labels, with cash and generic mobile payment as sensible defaults. Do not imply a payment gateway is integrated just because a method label exists.
- Support fully paid, partially paid and due sales. Payment allocation must be explicit and ledger-linked.
- Complete Sale runs one atomic transaction that stores sale header/items/payment, updates stock, records cash movements and updates customer receivable where applicable.
- Prevent accidental double submission. If a save fails, preserve the cart and show a recoverable error.
- After success, open the receipt screen. Provide actions to start another sale, view receipt, share/export a receipt image/PDF if implemented, and print only when a supported printer integration exists.
- Do not make a real printer or share feature look functional if it has not been implemented.

6.6 Receipt

- Clean print-friendly receipt with shop name/contact (if configured), invoice number, date/time, line items, quantities, unit prices, discount, total, paid, due and payment method.
- Invoice number must be unique within the business/device and robust across restarts and restores.
- QR code is optional and only used if it encodes a meaningful supported payload; do not display decorative nonfunctional QR codes.
- Share/export and print controls must reflect actual implementation and available Android capabilities.
- Receipt must use a compact, high-contrast layout and remain legible when exported or printed.

6.7 Inventory and stock movements

- Inventory summary: total product count, low-stock count and out-of-stock count.
- Filters: All, In Stock, Low Stock, Out of Stock. Filters must be correct and composable with search.
- Stock adjustment requires quantity, direction/reason and optional note. Record every adjustment in StockMovements.
- Purchase receiving increases stock only when the purchase/receiving transaction is confirmed. Draft purchases must not alter stock.
- Sales reduce stock; returns can increase stock only if the returned goods are marked as restockable.
- Never allow a silent direct stock overwrite without recording an adjustment event. Opening stock is a separately recorded opening movement.
- Use a consistent quantity model that supports fractional units where relevant, such as kilograms or litres, without floating-point rounding errors.

6.8 Customers and customer ledger

- Customer list supports search, add/edit, phone (optional), notes and current outstanding balance.
- Support walk-in customer sales without requiring a customer record.
- Customer details show transaction history, payments, returns and current due balance.
- Collect Due flow records an incoming payment against the customer and reduces receivable; it does not create a new sale.
- Allow partial payments and optional payment notes. Every ledger change must be traceable to its source transaction.
- Phone number is optional; do not require contact permissions just to enter a number manually.

6.9 Suppliers and purchases

- Supplier list: name, optional phone/contact, outstanding payable and purchase history.
- Purchase flow: supplier (optional for cash purchases), items, quantities, purchase costs, payment amount and remaining payable.
- Confirming a received purchase updates inventory and creates the corresponding payable/cash movement in one consistent transaction.
- Pay Supplier flow records a payment against outstanding payable; it must not create a duplicate expense or purchase.
- Supplier balance should be derived from linked transactions/ledger entries, not maintained as an independent editable number.

6.10 Expenses and cashbook

- Expense entry: category, amount, date/time, payment source, optional note and optional attachment if implemented.
- Common categories: rent, electricity, transport, salary, maintenance and miscellaneous. Users can add categories.
- Cashbook distinguishes cash-in and cash-out movements with source references. Transfers between cash/bank/mobile wallets must not be counted as income or expense.
- Show opening cash, cash received, cash paid out, expected closing cash and actual counted cash. Differences must be visible as reconciliation differences, not silently adjusted.

- Expense payments should affect the relevant cash account. An expense paid on credit, if supported, must be represented differently from a cash expense.
- Prevent the same expense from being entered twice due to double taps or retries.

6.11 Reports

- Date presets: today, 7 days, 30 days, 3 months and custom range where practical.
- Sales report: gross sales, discounts, returns, net sales, number of transactions and average sale value.
- Purchase report: purchases received, amount paid and amount outstanding.
- Expense report: totals by category and period.
- Profit & Loss: net sales less cost of goods sold, then recognized operating expenses; document the accounting assumptions used.
- Cash report: cash received, cash paid out, expected cash balance and reconciled actual cash if entered.
- Due report: receivables from customers and payables to suppliers.
- Charts must be based on stored records. With insufficient history, show a clear empty/limited-data state rather than misleading trends.
- CSV export should respect selected date range and use consistent column names, currency formatting and UTF-8 compatibility.

6.12 Settings / More

- Business profile: shop name, optional contact/address, receipt details and business type.
- Preferences: language (English/Bangla), theme (light by default; optional dark mode only after correct implementation), currency display and date formatting.
- Backup & Restore: export complete versioned backup, import and validate backup, show preview/warning before overwrite, and never silently replace data.
- Data export: CSV for products and reports where appropriate.
- Privacy Policy, Terms & Conditions and About/data handling pages must open reliably and contain actual relevant content; no blank pages.
- Help & Support: concise guidance, troubleshooting and data-safety explanation. Do not show fake support contact details.
- About page includes app version and acknowledgements for third-party libraries where required by licences.

7. Business Rules and Accounting Integrity

7.1 Financial concepts

Concept	Definition / rule
Gross sales	Sum of sale line totals before returns/adjustments according to the documented discount policy
Net sales	Sales after discounts and returns, according to a consistent policy
COGS	Cost assigned to the units actually sold; use a documented costing method, with weighted average as a practical MVP default
Gross profit	Net sales minus COGS
Net profit	Gross profit minus recognized operating expenses in the selected period; label estimates/limitations honestly
Cash received	Money actually received, not the full value of a credit sale
Receivable	Amount customers still owe
Payable	Amount the shop still owes suppliers
Cash balance	Opening balance plus recorded cash inflows minus recorded cash outflows; actual counted cash is separate

7.2 Transaction invariants

- A completed sale must have a stable ID, timestamp, item snapshots, totals, payment records and stock movements.
- Sale item snapshots preserve name, SKU, unit, unit price, cost basis and quantity at sale time.
- Every financial movement has a type, amount, date, source reference and account/method where applicable.
- Ledger balances must be reproducible from entries. Do not let users directly edit a derived balance without a documented opening-balance or adjustment transaction.
- Use database transactions for multi-table operations. On failure, roll back all related changes.
- Use idempotency protection for sale completion, payment recording and restore/import actions.
- Use explicit reversal/return/void records for completed transactions and retain audit history.
- Define and test policies for negative stock, returns, discounts, fractional quantities and price changes. Default to warning/blocking negative stock unless shop settings explicitly allow it.
- Store timestamps consistently and format them using the user's locale. Define the business-day boundary and use it consistently in every report.

8. Data Model and Persistence

Entity	Important fields / notes
BusinessProfile	id, name, shopType, contact/address optional, currency, createdAt
BusinessSettings	language, theme, stock policy, low-stock defaults, business-day preferences
Category	id, name, archivedAt optional
Unit	id, name, symbol, fractional support/precision
Product	id, name, categoryId, unitId, SKU, barcode, cost, selling price, threshold, archivedAt
ProductVariant	optional size/colour/serial/batch metadata; expand in later phase
StockMovement	id, productId, quantity delta, reason, source type/id, timestamp, note
Sale	id, invoice number, date, customerId optional, subtotal, discount, total, status, createdAt
SaleItem	id, saleId, productId, name/SKU/unit/price/cost snapshots, quantity, discount, line total
Payment	id, source transaction, amount, method/account, date, direction, reference
Purchase / PurchaseItem	supplier, received status, item/cost snapshots, quantities, totals and payment state
Customer / Supplier	id, name, optional phone/contact, notes, archive state
LedgerEntry	party type/id, amount, direction, source reference, date, memo
Expense	id, category, amount, date, payment account, note, status
CashAccount / CashMovement	account type/name, amount delta, source reference, timestamp
AuditEvent	event type, entity/source id, timestamp, summary; avoid unnecessary sensitive data
BackupMetadata	format version, export timestamp, app/schema version, integrity/checksum metadata

Use foreign keys and indexes for common search/filter paths. Define Room entities, DAOs and repositories deliberately. Use migrations for every schema change; never use destructive migration in a production build. Avoid storing derived balances in multiple places unless there is a clear reconciliation strategy.

9. Architecture and Code Quality

- Recommended stack: Kotlin, Jetpack Compose, Material 3 foundation with custom theme, ViewModel, Kotlin coroutines/Flow, Room/SQLite, DataStore for preferences and WorkManager only for appropriate deferred tasks.

- Use a layered structure: UI/presentation → domain use cases and validation → repository interfaces/implementations → Room DAOs and local storage.
- Keep accounting and stock calculations out of composables. UI should render state and dispatch events, not mutate the database directly.
- Use immutable UI state and explicit loading/success/error states. Avoid scattered mutable global state.
- Use dependency injection consistently. Manual dependency wiring is acceptable for a small MVP; avoid unnecessary architecture ceremony.
- Separate UI strings, date/currency formatting and domain logic so Bangla localization does not alter calculation logic.
- Use stable keys for lists and avoid recomposing the entire product catalog after every minor state update.
- Handle exceptions intentionally. Never swallow database errors or show success before persistence completes.
- Use clear package/module boundaries; do not create dozens of empty abstraction layers for appearance alone.

10. Localization, Accessibility and Device Support

- All user-facing strings must be externalized and translatable. Do not concatenate English fragments to create sentences.
- Support Bangla numerals only if selected as an explicit preference; otherwise maintain readable consistent number formatting. Currency is BDT by default and displayed consistently.
- Support long Bangla labels without clipping. Use flexible widths, wrapping where appropriate and responsive row layouts.
- Test font scaling and Android accessibility services. Provide content descriptions for meaningful icons; decorative icons should not be announced.
- Touch targets should be comfortably tappable (target around 48 dp where feasible). Maintain adequate text contrast and visible focus/pressed states.
- Support compact Android phones and tall screens. Do not depend on one fixed screenshot dimension or fixed-height content cards.
- Keep the first-run experience light and uncluttered. Do not force the user to configure advanced accounting before recording a first sale.

11. Security and Privacy

- Keep business data local in the MVP. Do not transmit it to third parties or add analytics/ads SDKs without an explicit product decision and clear disclosure.
- Request permissions only when needed and explain why. Core sales, inventory and ledger features require no location permission.
- Use Android's file picker/Storage Access Framework for user-directed backup import/export where suitable rather than broad storage access.
- Validate imported files and enforce size/format checks. Reject malformed or incompatible backups safely without overwriting existing data.
- Never hard-code secrets or API keys. MVP should not need them.
- Privacy Policy and Terms pages must accurately describe local data handling, backup limitations, permissions and third-party library use.

12. Backup, Restore and Data Portability

- Export a complete backup that includes all business entities, transaction history, stock movements, ledger entries, settings and schema/format version.

- Use a documented, versioned format with integrity validation. JSON or a carefully designed archive is acceptable; avoid an undocumented raw database copy as the only format.
- Before restore, validate the entire file and show the business name, export date, app/format version and the consequences of replacement or merge.
- Never restore partial data silently. If restore fails, existing data must remain intact.
- Provide an explicit confirmation before replacing local data. A safe MVP may support restore-to-empty/replacement first; merge can be added only when conflict rules are implemented.
- Provide CSV exports for useful user-owned data, but distinguish CSV exports from a complete restorable backup.
- Document that a backup file should be copied to a separate safe location; data on the same phone is not protected against device loss.

13. Performance and Reliability Targets

Area	Target / acceptance intent
Launch	Fast launch without network waiting or a fake loading screen
Common actions	Open Products, search, open New Sale and navigate between main tabs should feel immediate on a modest Android phone
Sale persistence	A completed sale is acknowledged only after its database transaction succeeds
Search	Indexed product/SKU/barcode search remains responsive for several thousand products
List rendering	Use lazy lists and pagination/windowing where necessary; avoid loading huge histories into memory
Offline operation	All MVP core flows function in airplane mode
Recovery	Force-stop/relaunch does not lose committed transactions
Database upgrades	Migration tests preserve existing records
Visual quality	No clipped text, overlapping controls, broken system-bar insets or unintended horizontal overflow

Do not claim hard performance guarantees without measuring on a real device. Profile and test using representative data before release.

14. Testing Strategy

14.1 Unit tests

- Totals, discounts, tax fields if implemented, fractional quantity rounding and currency precision.
- COGS, gross profit, net profit, returns and period boundaries.
- Partial payments, receivables, supplier payables and ledger reconciliation.
- Stock receiving, sale deduction, restocking returns and stock adjustment reasons.
- Idempotency and duplicate submit prevention.

14.2 Database/integration tests

- Sale writes all related records atomically.
- Failure injected during transaction rolls back all changes.
- Backup export/import round-trip preserves records and references.
- Migrations from every supported previous schema version.
- Archive product with historical sales without breaking receipt/report rendering.
- Restore validation fails safely for corrupt/incompatible backup.

14.3 UI and manual tests

- First launch, setup skip paths, empty states and first real sale.

- Product add/edit/search/filter, sale with multiple items, partial payment, collect due and return.
- All navigation actions, back navigation, form validation and unsaved-change warnings.
- English and Bangla on every screen, including long labels and error messages.
- Header placement on compact/tall screens, display cutout, gesture navigation, three-button navigation and keyboard open.
- Low-end device scrolling, repeated taps, process death and relaunch.
- Airplane-mode end-to-end scenario.

15. MVP Acceptance Checklist

- ☐ App installs and launches without account creation or internet.
- ☐ New shop setup saves and survives process death.
- ☐ User can add a product and stock appears correctly.
- ☐ User can complete a sale and stock decreases exactly once.
- ☐ Partial payment creates the correct cash movement and receivable.
- ☐ Collecting a due reduces receivable without creating another sale.
- ☐ Purchase receiving increases stock and supplier payable/cash is correct.
- ☐ Expense is recorded once and the correct cash account is affected.
- ☐ Returns and voids preserve audit history and correctly reverse applicable amounts/stock.
- ☐ Dashboard and reports derive from real local records, not mock constants.
- ☐ Profit report separates net sales, COGS, gross profit and expenses.
- ☐ Backup/restore round-trip works and corrupt restore leaves existing data intact.
- ☐ Database migrations preserve existing data.
- ☐ English/Bangla localization works across all screens.
- ☐ All top headers respect system insets and match the visual reference without excessive blank space.
- ☐ No visible dead buttons, fake loading states, placeholder pages or misleading feature claims.
- ☐ Core workflows pass on a real Android device in airplane mode.

16. Phased Implementation Plan for Google AI Studio

Give the builder one phase at a time. After each phase, build and test before proceeding. It must inspect the current code first and preserve completed work.

Phase	Scope	Exit condition
0. Audit & plan	Inspect project, framework, current files, dependencies, navigation, persistence and existing bugs. Report the implementation plan before editing.	No destructive changes; clear file-level plan
1. Design foundation	Theme, palette, typography, spacing, icons, shapes, system bars/insets, reusable fields/buttons/cards, navigation shell.	Header/safe-area checks pass; 3 reference screens visually tuned
2. Local data foundation	Room schema, migrations, repositories, use cases, business setup, sample/demo mode distinction.	Persistence and migration tests pass
3. Products & inventory	Product forms, search/filter, stock movements, low-stock alerts.	Create/edit/search and stock tests pass
4. Sales & receipt	Cart, checkout, payments, transaction-safe stock updates, receipts, returns.	End-to-end sale and rollback tests pass
5. Ledger & cash	Customers, suppliers, purchases, due collection, expenses and cashbook.	Reconciliation scenarios pass
6. Reports	Sales, expenses, cash, dues, COGS and profit/loss reports.	Reports reconcile with source transactions
7. Safety & localization	Backup/restore, CSV, Bangla/English, permissions, privacy/legal screens.	Round-trip backup and localization checks pass

8. Hardening	Performance, accessibility, edge cases, visual regression, release build.	Acceptance checklist complete

17. Rules for Every AI Studio Iteration

- At the start, summarize the current implementation and identify what is already complete. Do not rebuild completed features unnecessarily.
- Make changes only within the requested phase. Do not sneak in a redesign, new feature, paid service or unrelated refactor.
- Before editing, identify relevant files and dependencies. After editing, list files changed, explain behaviour, and report build/test status honestly.
- If a dependency is unavailable or incompatible, explain it and choose a maintained free alternative; do not invent API calls.
- Never say a feature is complete merely because a screen exists. Confirm the underlying data flow and interactions.
- Do not replace a working local database with mock arrays, in-memory state or fake repository implementations.
- Do not silently remove features, reset user data, overwrite schema history or change branding/theme without request.
- If an instruction is ambiguous, choose the safest implementation consistent with this PRD and the reference, and clearly state the assumption.
- Keep prompts and implementation tasks phase-sized. Do not ask AI Studio to generate the entire app in one giant uncontrolled response.

18. Explicitly Out of Scope for MVP

- Mandatory cloud account or Google login.
- Paid AI models, paid APIs or paid backend services.
- Real-time multi-device sync or multi-branch cloud management.
- Integrated payment gateway or automatic mobile wallet reconciliation.
- Complex statutory accounting, tax filing or jurisdiction-specific compliance guarantees.
- Advanced payroll, attendance and role-based staff permission system.
- Automatic WhatsApp/SMS reminders requiring paid gateways.
- Advanced serial/IMEI, batch/expiry, variants and supplier purchase order workflows until core MVP is stable.

19. Final Definition of Done

ShopOS is ready for its first practical release only when the main product, sales, stock, payment, ledger, expense, reporting and backup flows operate on real local data; transaction calculations are tested; core workflows work offline; old data survives app restarts and database migrations; all controls work; English/Bangla strings are handled correctly; and the UI matches the approved mockup with clean top-header/safe-area placement.

Visual polish is not a substitute for data integrity, and data integrity is not an excuse for generic UI. Both are release requirements.

Appendix A. Compact Master Prompt for AI Studio

Build ShopOS strictly according to this complete PRD. First inspect the existing project and report its framework, architecture, current functionality and safe implementation plan. Implement only Phase 1 unless a different phase is explicitly requested. The attached approved pastel lavender/periwinkle, pearl-white and blush mockup is the primary UI

reference: reproduce its composition, typography, spacing, cards, lists, tabs, bottom navigation, icon style and overall premium visual language as closely as practical. Do not substitute a generic Material dashboard. Carefully implement Android system bars and WindowInsets: apply top safe inset once, keep page headers compact and directly below the actual status bar, avoid duplicate top padding or arbitrary spacers, and test cutouts, compact/tall devices, keyboard and both navigation modes. Use Kotlin, Jetpack Compose, Room and a layered architecture unless the existing project requires a documented alternative. Preserve all existing working features and data; no destructive schema reset, no fake business data in real mode, no paid API, no mandatory login, and no internet requirement for core workflows. All visible controls must work. Money, stock, payments, ledgers and reports must be transaction-safe and tested. Build incrementally, compile/test after each phase, and report exactly what is complete versus still pending.